Data-Driven Forecasting: Car MSRP

1st Da Zhang
MADS
University of Victoria
dazhang@uvic.ca

2nd Yuanming Wang
MADS
University of Victoria
yuanmingwang@uvic.ca

3rd Zhenwei Pan
MTIS
University of Victoria
zhenweipan@uvic.ca

4th Christopher Xu
MSc
University of Victoria
pengyuanxu@uvic.ca

5th Etta Zhang
MADS
University of Victoria
Xinyuzhang8@uvic.ca

***Abstract*—This project investigates data-driven forecasting of car manufacturer's suggested retail price (MSRP) using a heterogeneous vehicle dataset from Kaggle's Car Price Prediction Challenge, containing over 19,000 cars with mixed numerical and categorical attributes. We first perform data cleaning to correct corrupted manufacturer and model names, coerce textual numerics to proper types, and handle missing values. We then engineer domain-motivated features such as brand region (BrandCountry), vehicle age, mileage intensity, log-scaled engine and mileage terms, premium-brand indicators, and interaction features that capture non-linear relationships between usage, age, and engine size.**

Using this processed dataset, we frame MSRP prediction as a supervised regression problem and construct a unified preprocessing pipeline combining imputation, scaling, and one-hot encoding. On this common feature space, we benchmark several models: a naive mean-price baseline, ridge-regularised linear regression, LightGBM gradient-boosted trees, and CatBoost. CatBoost emerges as the best model, reducing test RMSE by roughly 40% compared with the naive baseline and clearly outperforming ridge and LightGBM, especially for non-luxury vehicles. Residual and diagnostic analyses show that the model fits the bulk of the market well, with remaining errors concentrated in the sparse, very high-end segment. Overall, the study demonstrates an end-to-end workflow—from SQL-based exploration and feature engineering to modern gradient-boosting regression—for accurate and interpretable car price prediction.

***Index Terms*—car price prediction, CatBoost, random forest, regression, feature engineering, SQL analytics**

I. Introduction

Accurate car price prediction is a key problem in automotive analytics, as pricing directly reflects both product value and market positioning. Traditional valuation often relies on expert judgment, which can be subjective and inconsistent. With the growth of data availability, data-driven analysis provides an objective way to understand how technical and market features—such as engine capacity, horsepower, fuel type, mileage, and brand origin—affect vehicle prices. Using structured data and SQL-based exploration, patterns and correlations among these attributes can be revealed before developing predictive models.

This project focuses on building and comparing several regression-based car price prediction models (including multiple linear regression, Random Forest, LightGBM, and CatBoost) to quantify how each feature contributes to price variation.

II. Data

A. Data Description

1) source of data: The data was collected from the publicly available Kaggle dataset "Car Price Prediction Challenge" (https://www.kaggle.com/datasets/deepcontractor/car-price-prediction-challenge). The dataset aggregates vehicle information from multiple manufacturers and markets, making it suitable for regression-based price modeling and comparative analysis.

2) Number of rows (tuples), and number of features (attributes): The dataset includes 19,237 rows and 18 attributes, with each row representing an individual car.

3) training vs. testing split : To evaluate the generalization performance of the predictive model, the dataset was randomly divided into 70% (13,466 rows) for training and 30% (5,771 rows) for testing. The split ensured that samples were proportionally distributed across major features such as fuel type, brand, and body category. The training data was used to build and tune the model, while the testing set was reserved for performance validation.

4) Target Response variable: The target response variable is Price, representing the car's market value in USD.

5) Feature variables:

a) Numerical variables: Price, Levy, Engine volume, Mileage, Cylinders, and Airbags.

b) Categorical variables: Manufacturer, Model, Prod year, Category, Leather interior, Fuel type, Gear box type, Drive wheels, Doors, Wheel, and Color.

B. Data cleaning and preprocessing

1) Feature tuples syntax errors (Model, Manufacturer):

2) Feature creation and transformation [Manufacturers - BrandCountry]: A new categorical feature, BrandCountry, was created to enrich the dataset and enable region-based analysis. The original data did not include information about the country or region of each manufacturer, which limited the ability to compare pricing patterns across different markets. To address this, a dictionary mapping was constructed to link each manufacturer to its country or regional origin. For example, Toyota, Honda, and Nissan were assigned as Asian; Ford and Chevrolet as American; and BMW, Mercedes-Benz, and Audi as European.

III. SQL

A. Q1: How does the gasoline percentage in the market compared to year 2000 to after year 2000? [SQL Query]

a) Result:

TABLE I
THE GASOLINE MARKET SHARE BEFORE AND AFTER THE YEAR 2000

Period	gasoline
After 2000	73.64
Before 2000	75.18

B. Q2: Is there any difference between the average car prices across different brand countries (Asian, American, European)? [SQL Query]

a) Result:

TABLE II
COMPARISON OF AVERAGE CAR PRICES ACROSS BRAND COUNTRIES

Brand_Country	Avg_Price
American	15442.29398
Asian	17366.15844
European	23102.22666

C. Q3: What models does MERCEDES-BENZ make? [Neo4j]

a) Result: Fig. 1.

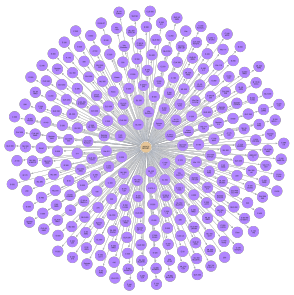

Fig. 1. Models made by MERCEDES-BENZ

D. Q4: Which car features tend to co-occur in high-priced models? [Neo4j] We define high-priced as the top 20% most expensive, which is 25559.0 in our data.

a) Result: Fig. 2.

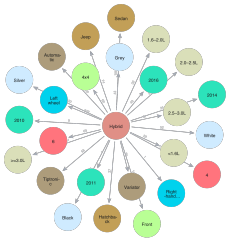

Fig. 2. High-priced car feature co-occurrence with Hybrid

IV. PREDICTION ANALYSIS AND DISCUSSION

Before constructing our full modeling pipeline in Section V, we first performed a set of preliminary baseline experiments to understand how well standard regression methods capture pricing patterns in the cleaned dataset. These baselines serve two purposes: (1) they provide interpretable reference points for later models, and (2) they confirm that car prices exhibit strong nonlinear relationships that simple linear models cannot capture.

A. Implementation

We first implemented a multiple linear regression model as the simplest possible baseline. To capture nonlinear dependencies between mileage, engine size, production year, and market attributes, we then trained a Random Forest Regressor using the same cleaned and encoded features. Categorical variables were one-hot encoded, while numeric features were kept continuous.

This early modeling stage is intentionally lightweight—its goal is to determine whether nonlinearity matters and whether tree-based models provide measurable improvements before moving to more advanced gradient-boosting methods.

B. Result

Across these preliminary models, Random Forest consistently achieved lower error than linear regression, indicating that nonlinear interactions (e.g., heavy mileage interacting with older production years) are essential for explaining price variation. The model produced reasonably accurate predictions for typical vehicles but showed larger errors for very expensive or uncommon models, where the dataset is sparse.

These patterns motivate the need for a more systematic modeling pipeline with richer feature engineering and more advanced algorithms. Section V therefore extends these baselines using Ridge regression, LightGBM, and CatBoost.

C. Discussion

Random Forest confirms several key insights for the larger modeling workflow:

- Nonlinear relationships dominate price behaviour.
- Prediction error varies strongly across the price spectrum.
- Very high-end vehicles remain difficult due to sparse data and missing attributes.

However, the model alone is not sufficient for precise MSRP estimation. This motivates the gradient-boosting approach of Section V, where CatBoost ultimately provides substantially better accuracy and stability. Due to the presence of random, inconsistent, and garbled text entries across multiple attributes, the resulting noise can have a substantial negative impact on

model performance. These irregularities reduce the reliability of categorical features and introduce variability that is difficult to standardize within the scope of this project. For example, the model name field contains variations such as "Audi A3," "3 sport," and "A3 sport," which refer to similar vehicle types but cannot be easily normalized without extensive domain knowledge or manual intervention. This level of inconsistency degrades the signal quality of the dataset, leading to underfitting and ultimately reducing the model's predictive accuracy while increasing error magnitudes. A future extension of this project could involve implementing more advanced text-cleaning and entity-resolution method. Potential approaches include Natural language processing (NLP) based text normalization, active learning or semi-supervised labeling. These enhancements would significantly reduce noise in the dataset, improve feature consistency, and likely yield meaningful gains in model performance in prediction.

V. Modeling Methodology

Building on the earlier SQL-based exploration and the preliminary linear regression and Random Forest baselines, we now describe our final modeling pipeline. This pipeline introduces more systematic feature engineering, a unified preprocessing workflow, and a comparison of several regression models (naive mean baseline, ridge regression, LightGBM, and CatBoost), from which CatBoost emerges as the best trade-off between accuracy and interpretability.

A. Overview

We frame MSRP prediction as a supervised regression problem. Starting from the cleaned Kaggle car dataset, our pipeline consists of:

1) Data cleaning and domain-driven feature engineering.
2) Splitting the data into training and test sets.
3) Preprocessing numerical and categorical variables with a unified `scikit-learn` pipeline.
4) Training and comparing several models:
   a) a naive mean-price baseline,
   b) ridge-regularised linear regression,
   c) LightGBM gradient-boosted trees, and
   d) CatBoost gradient-boosted trees.
5) Selecting CatBoost as the final model and analysing its performance in detail.

The preprocessing and feature-engineering steps are shared by all models; only the final estimator changes.

B. Data Cleaning and Feature Engineering

From the cleaned CSV, we apply additional processing:

- Remove duplicate rows based on the unique `id`.
- Coerce key columns to numeric types: `Price`, `Levy`, `Mileage`, `Engine_volume`, `Prod_year`, `Cylinders`, and `Airbags`.
- For textual numeric fields (e.g., mileage or levy stored as strings), strip non-digit characters and convert to numbers; invalid parses are set to missing.
- Map each `Manufacturer` to a coarse region (American, European, Asian, Other) to form a `BrandCountry` feature; unknown brands are mapped to "Other".
- Drop rows with missing values in core variables (`Price`, `Prod_year`, `Mileage`, `Engine_volume`) to keep the target and main predictors reliable.
- Winsorise `Price`, `Levy`, and `Mileage` at the 1st and 99th percentiles to reduce the impact of extreme outliers.

We then add several engineered features:

- **`Doors_numeric`**: numeric door count extracted from the `Doors` string.
- **`Prod_year_bin`**: 5-year buckets of `Prod_year`.
- **`Vehicle_age`**: maximum production year in the dataset minus `Prod_year`, clipped at 0.
- **`Mileage_log`**: $\log(1 + \texttt{Mileage})$.
- **`Mileage_per_year`**: $\texttt{Mileage}/\max(\texttt{Vehicle\_age}, 1)$.
- **`Mileage_per_year_log`**: $\log(1 + \texttt{Mileage\_per\_year})$.
- **`Engine_volume_log`**: $\log(1 + \texttt{Engine\_volume})$.
- **`Engine_per_cylinder`**: `Engine_volume`/`Cylinders` for positive cylinder counts.
- **`Is_premium_brand`**: binary flag for premium brands (e.g., BMW, Mercedes-Benz, Audi, Porsche).
- **`Model_frequent`**: frequency count of each `Model` in the dataset.
- **Cyclical encoding of production year**:

$$\texttt{Prod\_year\_sin} = \sin(2\pi \cdot \texttt{year\_norm}),$$

$$\texttt{Prod\_year\_cos} = \cos(2\pi \cdot \texttt{year\_norm}),$$

  where `year_norm` is the min–max normalised production year.
- **Interaction features:**

$$\texttt{Age\_x\_Mileage\_log} = \texttt{Vehicle\_age} \cdot \texttt{Mileage\_log},$$

$$\texttt{Premium\_x\_Engine\_log} = \texttt{Is\_premium\_brand} \cdot \texttt{Engine\_volume\_log},$$

$$\texttt{Age\_per\_cylinder} = \frac{\texttt{Vehicle\_age}}{\texttt{Cylinders}}$$

for positive cylinder counts.

These engineered columns are used together with original categorical features such as `Manufacturer`, `Model`, `Category`, `Leather_interior`, `Fuel_type`, `Gear_box_type`, `Drive_wheels`, `Wheel`, `Color`, and `BrandCountry`.

C. Train–Test Split and Preprocessing

We use the raw `Price` in USD as the target variable. The dataset is split into 70% training and 30% test using a fixed random seed (`random_state = 42`), resulting in approximately 13,245 training examples and 5,677 test examples for the CatBoost runs.

We build a `scikit-learn ColumnTransformer` to preprocess numeric and categorical columns:

- **Numeric features** (original and engineered numeric columns): median imputation followed by `StandardScaler` (zero mean, unit variance).
- **Categorical features** (`Manufacturer`, `Model`, `Category`, `Leather_interior`, `Fuel_type`, `Gear_box_type`, `Drive_wheels`, `Prod_year_bin`, `Wheel`, `Color`, `BrandCountry`): most-frequent imputation followed by `OneHotEncoder` with `handle_unknown="ignore"`.

The resulting design matrix is then used by all downstream models.

D. Models Considered

We compare four models:

1) Naive Mean Baseline: The naive baseline always predicts the mean training price for all cars. This provides a simple reference level, with performance:

$$\text{MSE} \approx 2.58 \times 10^8, \qquad \text{RMSE} \approx \$16{,}073,$$
$$\text{MAE} \approx \$11{,}757, \qquad R^2 \approx -0.0005.$$

2) Ridge-Regularised Linear Regression: We train a linear model on the preprocessed features, but fit on log-transformed prices $\log(1 + \texttt{Price})$, and then map predictions back via $\exp(\widehat{y}_{\log}) - 1$. The L2 penalty α is tuned via 5-fold cross-validation on the training set.

Ridge improves over the baseline but still underfits the complex non-linear relationships in the data, with approximate test performance:

$$\text{MSE} \approx 1.78 \times 10^8, \qquad \text{RMSE} \approx \$12{,}578,$$
$$\text{MAE} \approx \$7{,}748, \qquad R^2 \approx 0.57.$$

3) LightGBM Gradient-Boosted Trees: LightGBM is a tree-based gradient boosting method applied to the same preprocessed features. We run a small hyperparameter search over learning rate, number of leaves, maximum depth, and number of estimators.

In our setting, LightGBM performs comparably to ridge but does not clearly outperform CatBoost, so we keep it as a secondary baseline rather than the final model of choice.

4) CatBoost Gradient-Boosted Trees (Final Model): CatBoost is particularly well suited for datasets with many categorical features and heterogeneous feature scales. We use `CatBoostRegressor` with RMSE loss, GPU acceleration, and an explicit list of categorical features so that CatBoost can apply its internal target statistics encoding (while combining this with our one-hot encoding to mitigate leakage from rare categories).

We perform a grid search over 16 hyperparameter combinations, varying:

- learning rate,
- tree depth,
- number of iterations,
- L_2 leaf regularisation.

The best configuration (selected by 5-fold cross-validated RMSE on the training set) uses:

- learning rate $= 0.03$,
- depth $= 8$,
- iterations $= 2000$,
- L_2 leaf regularisation $= 3.0$.

All models are trained on the same train–test split to allow fair comparison.

E. Evaluation Protocol and Metrics

For each model we evaluate performance on the held-out test set using:

- Mean Squared Error (MSE),
- Root Mean Squared Error (RMSE),
- Mean Absolute Error (MAE),
- Coefficient of determination R^2,
- A robustified Mean Absolute Percentage Error (MAPE), where the price is floored at \$5,000 in the denominator to avoid exploding errors for very cheap cars.

Because very expensive cars dominate squared errors, we also mark the top 5% of the test set by price as outliers (those with prices above \$66,485). We report metrics both:

- on the full test set, and
- on the non-outlier subset (prices $\leq \$66{,}485$).

F. Final CatBoost Model and Diagnostics

CatBoost clearly outperforms all other models. Its approximate performance is:

a) Full test set (CatBoost).:

$$\text{MSE} \approx 9.54 \times 10^7, \qquad \text{RMSE} \approx \$9{,}766,$$
$$\text{MAE} \approx \$5{,}381, \qquad R^2 \approx 0.63.$$

b) Non-outlier subset (CatBoost).:

$$\text{MSE} \approx 7.07 \times 10^7, \qquad \text{RMSE} \approx \$8{,}406,$$
$$\text{MAE} \approx \$5{,}077, \qquad R^2 \approx 0.61,$$

Overall, CatBoost reduces RMSE by about 40% compared with the naive mean predictor and still yields a substantial improvement over ridge and LightGBM on the same feature space.

To understand model behaviour, we examine several diagnostic plots :

c) Actual vs. predicted prices.: Most test points lie close to the 45-degree line for prices up to about \$60,000, indicating a good fit for typical cars. However, very expensive vehicles (above \$100,000) are systematically under-predicted: their true prices sit far above the diagonal. This suggests that the model struggles with rare luxury cars, where important factors such as options, condition, or fine-grained trim differences within a model line are not fully captured by the available features.

d) Residual distribution.: The residual histogram is sharply peaked around zero, with a slightly heavy left tail. The bulk of residuals are within $\pm\$10{,}000$, but a small number of large negative residuals remain (true price substantially above prediction), again corresponding to the under-predicted luxury segment.

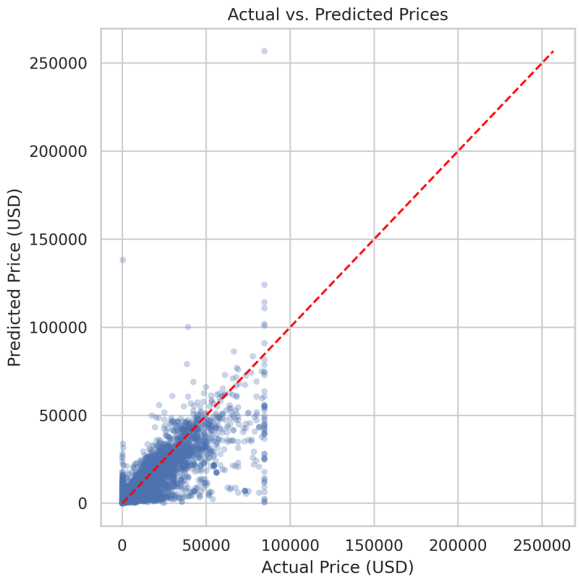

Fig. 3. For the bulk of the market (up to $60k), predicted prices lie close to the 45-degree line, while the most expensive vehicles are systematically under-predicted.

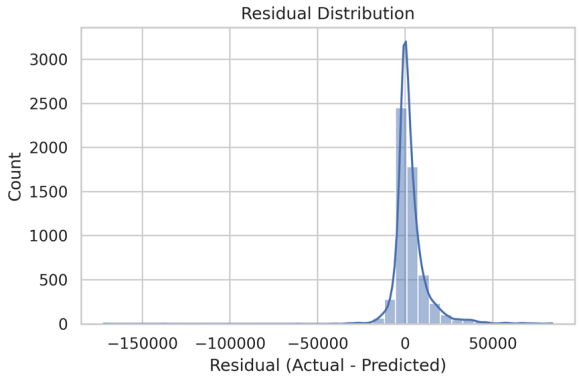

Fig. 4. Residuals sharply peaked around zero with a long left tail, again indicating a small number of high-price cars the model cannot fully capture.

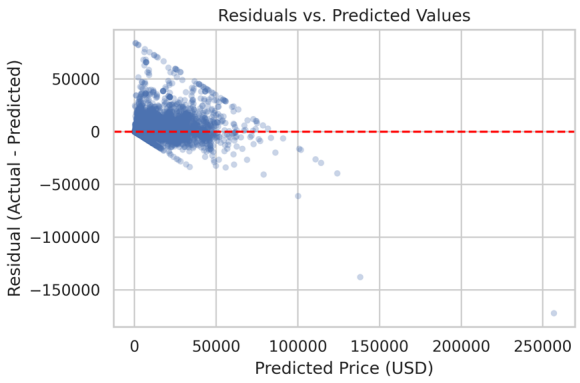

Fig. 5. Residuals are concentrated near zero for most cars, with a slight funnel shape: residual variance grows with predicted price and the few very expensive cars tend to be underestimated (large negative residuals).

e) Residuals vs. predicted values.: The residual–prediction plot shows a mild funnel shape: residual variance increases with predicted price. For low- and mid-priced cars, residuals are tightly clustered, while higher predicted prices are associated with more spread and more frequent under-estimation. This pattern indicates some heteroscedasticity and suggests that price-dependent loss weighting or segment-specific models could further improve performance.

f) Cross-validation stability.: Cross-validation error bars show that per-fold MSE values are all close to the mean (around 9.6×10^7), corresponding to RMSEs in the $9k–$10k range. The small variance across folds suggests that the CatBoost model is stable with respect to the specific train–test split and that the final test performance is not due to a lucky partition of the data.

VI. Appendix

A. SQL Code

```
SELECT
    CASE
        WHEN Prod_year <= 2000 THEN 'Before 2000'
        ELSE 'After 2000'
    END AS Period,
    ROUND(
        (SUM(
            CASE
                WHEN Fuel_type IN ('Petrol',
                    'Diesel') THEN 1
                ELSE 0
            END
        ) * 100.0) /
        COUNT(*),
        2
    ) AS Gasoline_Percentage
FROM Car
GROUP BY Period;


CREATE VIEW CarUpdate AS
    SELECT *,
        CASE
            WHEN Manufacturer = 'ACURA' THEN 'Asian'
            WHEN Manufacturer = 'ALFA ROMEO' THEN
                'European'
            WHEN Manufacturer = 'ASTON MARTIN' THEN
                'European'
            WHEN Manufacturer = 'AUDI' THEN
                'European'
            WHEN Manufacturer = 'BENTLEY' THEN
                'European'
            WHEN Manufacturer = 'BMW' THEN
                'European'
            WHEN Manufacturer = 'BUICK' THEN
                'American'
            WHEN Manufacturer = 'CADILLAC' THEN
                'American'
            WHEN Manufacturer = 'CHEVROLET' THEN
                'American'
            WHEN Manufacturer = 'CHRYSLER' THEN
                'American'
            WHEN Manufacturer = 'CITROEN' THEN
                'European'
            WHEN Manufacturer = 'DAEWOO' THEN
                'Asian'
            WHEN Manufacturer = 'DAIHATSU' THEN
                'Asian'
```

```
WHEN Manufacturer = 'DODGE' THEN
    'American'
WHEN Manufacturer = 'FERRARI' THEN
    'European'
WHEN Manufacturer = 'FIAT' THEN
    'European'
WHEN Manufacturer = 'FORD' THEN
    'American'
WHEN Manufacturer = 'GAZ' THEN
    'European'
WHEN Manufacturer = 'GMC' THEN
    'American'
WHEN Manufacturer = 'GREATWALL' THEN
    'Asian'
WHEN Manufacturer = 'HAVAL' THEN 'Asian'
WHEN Manufacturer = 'HONDA' THEN 'Asian'
WHEN Manufacturer = 'HUMMER' THEN
    'American'
WHEN Manufacturer = 'HYUNDAI' THEN
    'Asian'
WHEN Manufacturer = 'INFINITI' THEN
    'Asian'
WHEN Manufacturer = 'ISUZU' THEN 'Asian'
WHEN Manufacturer = 'JAGUAR' THEN
    'European'
WHEN Manufacturer = 'JEEP' THEN
    'American'
WHEN Manufacturer = 'KIA' THEN 'Asian'
WHEN Manufacturer = 'LAMBORGHINI' THEN
    'European'
WHEN Manufacturer = 'LANCIA' THEN
    'European'
WHEN Manufacturer = 'LAND␣ROVER' THEN
    'European'
WHEN Manufacturer = 'LEXUS' THEN 'Asian'
WHEN Manufacturer = 'LINCOLN' THEN
    'American'
WHEN Manufacturer = 'MASERATI' THEN
    'European'
WHEN Manufacturer = 'MAZDA' THEN 'Asian'
WHEN Manufacturer = 'MERCEDES-BENZ'
    THEN 'European'
WHEN Manufacturer = 'MERCURY' THEN
    'American'
WHEN Manufacturer = 'MINI' THEN
    'European'
WHEN Manufacturer = 'MITSUBISHI' THEN
    'Asian'
WHEN Manufacturer = 'MOSKVICH' THEN
    'European'
WHEN Manufacturer = 'NISSAN' THEN
    'Asian'
WHEN Manufacturer = 'OPEL' THEN
    'European'
WHEN Manufacturer = 'PEUGEOT' THEN
    'European'
WHEN Manufacturer = 'PONTIAC' THEN
    'American'
WHEN Manufacturer = 'PORSCHE' THEN
    'European'
WHEN Manufacturer = 'RENAULT' THEN
    'European'
WHEN Manufacturer = 'ROLLS-ROYCE' THEN
    'European'
WHEN Manufacturer = 'ROVER' THEN
    'European'
WHEN Manufacturer = 'SAAB' THEN
    'European'
WHEN Manufacturer = 'SATURN' THEN
    'American'
WHEN Manufacturer = 'SCION' THEN 'Asian'
WHEN Manufacturer = 'SEAT' THEN
    'European'
WHEN Manufacturer = 'SKODA' THEN
    'European'
WHEN Manufacturer = 'SSANGYONG' THEN
    'Asian'
WHEN Manufacturer = 'SUBARU' THEN
    'Asian'
WHEN Manufacturer = 'SUZUKI' THEN
    'Asian'
WHEN Manufacturer = 'TESLA' THEN
    'American'
WHEN Manufacturer = 'TOYOTA' THEN
    'Asian'
WHEN Manufacturer = 'UAZ' THEN
    'European'
WHEN Manufacturer = 'VAZ' THEN
    'European'
WHEN Manufacturer = 'VOLKSWAGEN' THEN
    'European'
WHEN Manufacturer = 'VOLVO' THEN
    'European'
WHEN Manufacturer = 'ZAZ' THEN
    'European'
END AS Brand\_Country
FROM Car;

-- SELECT * FROM CarUpdate;

SELECT Brand_Country, AVG(Price) AS Avg_Price
FROM CarUpdate
GROUP BY Brand_Country
ORDER BY Brand_Country;

DROP VIEW CarUpdate;
```

B. Cyoher Code

```
//// ---------- Create table ----------
// Constraints
CREATE CONSTRAINT car_id_unique IF NOT EXISTS
FOR (c:Car) REQUIRE c.ID IS UNIQUE;

CREATE CONSTRAINT manufacturer_name_unique IF NOT
    EXISTS
FOR (m:Manufacturer) REQUIRE m.name IS UNIQUE;

CREATE CONSTRAINT model_name_unique IF NOT EXISTS
FOR (mo:Model) REQUIRE mo.name IS UNIQUE;

// Import in batches
LOAD CSV WITH HEADERS FROM
    'file:///car_price_prediction_CLEANED.csv' AS
    row
CALL {
  WITH row

  MERGE (c:Car {ID: row.ID})
  SET  c.Price             = row.Price,
       c.Levy              = row.Levy,
       c.Manufacturer      = row.Manufacturer,
       c.Model             = row.Model,
       c.Prod_year         = row.Prod_year,
       c.Category          = row.Category,
       c.Leather_interior  = row.Leather_interior,
       c.Fuel_type         = row.Fuel_type,
       c.Engine_volume     = row.Engine_volume,
       c.Mileage           = row.Mileage,
       c.Cylinders         = row.Cylinders,
       c.Gear_box_type     = row.Gear_box_type,
       c.Drive_wheels      = row.Drive_wheels,
       c.Doors             = row.Doors,
       c.Wheel             = row.Wheel,
       c.Color             = row.Color,
```

```
      c.Airbags           = row.Airbags

  MERGE (m:Manufacturer {name: row.Manufacturer})
  MERGE (mo:Model {name: row.Model})
  MERGE (m)-[:MAKES]->(mo)
  MERGE (mo)-[:INSTANCES]->(c)
} IN TRANSACTIONS OF 10000 ROWS;

//// ---------- Q3 ----------
MATCH p=(:Manufacturer {name:
    "MERCEDES-BENZ"})-[:MAKES]->(:Model)
RETURN p;


//// ---------- Q4 ----------
//// 1. Make sure Price is numeric
MATCH (c:Car)
WHERE c.Price IS NOT NULL
SET c.priceNum = toFloat(c.Price);

// sanity-check
MATCH (c:Car)
RETURN c.Price, c.priceNum
LIMIT 10;

//// 2. Compute the top 20\% price threshold, finds
    the price at the 80th percentile (top 20\% most
    expensive)
CALL {
  MATCH (c:Car)
  WHERE c.priceNum IS NOT NULL
  WITH c
  ORDER BY c.priceNum DESC
  WITH collect(c.priceNum) AS prices
  // 0.2 = top 20% (80th percentile index)
  RETURN prices[toInteger(size(prices) * 0.2)] AS
      highPriceThreshold
}
RETURN highPriceThreshold;

//// 3. Create CO_OCCUR_HIGHPRICE relationships
    (top 20% cars)
// Clear old co-occurrence relationships (if any)
MATCH ()-[r:CO_OCCUR_HIGHPRICE]-()
DELETE r;
// 1) Compute 80th percentile price as the
    high-price threshold
CALL {
  MATCH (c:Car)
  WHERE c.priceNum IS NOT NULL
  WITH c
  ORDER BY c.priceNum DESC
  WITH collect(c.priceNum) AS prices
  RETURN prices[toInteger(size(prices) * 0.2)] AS
      highPriceThreshold
}
WITH highPriceThreshold

// 2) Take only high-priced cars (>= threshold)
MATCH (c:Car)
WHERE c.priceNum >= highPriceThreshold

// 3) Collect all feature nodes attached to each car
MATCH
    (c)-[:HAS_FUEL|IN_CATEGORY|HAS_GEARBOX|HAS_DRIVE|HAS_COLOR|
          HAS_WHEEL|MADE_IN|HAS_DOORS|HAS_CYLINDERS|ENGINE_BUCKET]->(f)
WITH c, collect(DISTINCT f) AS feats

// 4) Generate all unordered feature pairs per car
UNWIND feats AS f1
UNWIND feats AS f2
WITH c, f1, f2
WHERE id(f1) < id(f2)

// 5) Count how many high-priced cars share each
    pair
WITH f1, f2, count(DISTINCT c) AS coCount
MERGE (f1)-[r:CO_OCCUR_HIGHPRICE]->(f2)
SET  r.count = coCount;
```

For the complete SQL, Cypher and Modeling scripts, see the GitHub repository Algorithms-DataModels.